

Network Working Group
Internet-Draft
Intended status: Informational
Expires: 18 October 2026

C. Celi, Ed.
16 April 2026

ACVP ML-DSA JSON Specification
draft-celi-acvp-ml-dsa-01

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 October 2026.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

Table of Contents

1. Acknowledgements	2
2. Abstract	3
3. Introduction	3
4. Conventions	3
4.1. Notation conventions	3
4.2. Terms and Definitions	3
4.2.1. Prompt	3
4.2.2. Registration	4
4.2.3. Response	4

Celi	Expires 18 October 2026	[Page 1]
Internet-Draft	ACVP ML-DSA	April 2026

4.2.4. Test Case	4
4.2.5. Test Group	4
4.2.6. Test Vector Set	4
4.2.7. Validation	4
5. Supported ML-DSA Algorithms	4
6. Test Types and Test Coverage	4
6.1. Test Types	5
6.1.1. ML-DSA KeyGen Test Types	5
6.1.2. ML-DSA SigGen Test Types	5

6.1.3. ML-DSA SigVer Test Types	12
6.2. Test Coverage	13
6.2.1. Requirements Covered	13
6.2.2. Requirements Not Covered	13
7. Capabilities Registration	13
7.1. Prerequisites	13
7.2. Required Prerequisite Algorithms for ML-DSA Validations	14
7.3. ML-DSA keyGen Registration Properties	15
7.3.1. ML-DSA keyGen Mode Capabilities Example	16
7.4. ML-DSA sigGen Registration Properties	16
7.4.1. ML-DSA sigGen Mode Capabilities Example	18
7.5. ML-DSA sigVer Registration Properties	20
7.5.1. ML-DSA sigVer Mode Capabilities Example	22
8. Test Vectors	24
8.1. ML-DSA keyGen Test Vectors	25
8.1.1. ML-DSA keyGen Test Groups JSON Schema	25
8.1.2. ML-DSA keyGen Test Case JSON Schema	26
8.2. ML-DSA sigGen Test Vectors	27
8.2.1. ML-DSA sigGen Test Groups JSON Schema	27
8.2.2. ML-DSA sigGen Test Case JSON Schema	28
8.3. ML-DSA sigVer Test Vectors	31
8.3.1. ML-DSA sigVer Test Groups JSON Schema	31
8.3.2. ML-DSA sigVer Test Case JSON Schema	32
9. Test Vector Responses	35
9.1. ML-DSA keyGen Test Vector Responses	36
9.2. ML-DSA sigGen Test Vector Responses	37
9.3. ML-DSA sigVer Test Vector Responses	38
10. Security Considerations	39
11. IANA Considerations	39
12. Normative References	39
Author's Address	40

1. Acknowledgements

There are no acknowledgements.

Celi Expires 18 October 2026 [Page 2]

Internet-Draft ACVP ML-DSA April 2026

2. Abstract

This document defines the JSON schema for testing Module Lattice-based Digital Signature Algorithm (ML-DSA) implementations with the ACVP specification.

3. Introduction

The Automated Crypto Validation Protocol (ACVP) defines a mechanism to automatically verify the cryptographic implementation of a software or hardware crypto module. The ACVP specification defines how a crypto module communicates with an ACVP server, including crypto capabilities negotiation, session management, authentication, vector processing and more. The ACVP specification does not define algorithm specific JSON constructs for performing the crypto validation. A series of ACVP sub-specifications define the constructs for testing individual crypto algorithms. Each sub-specification addresses a specific class of crypto algorithms. This sub-specification defines the JSON constructs for testing Module Lattice-based Digital Signature Algorithm (ML-DSA) implementations using ACVP.

This document is a work-product of the NIST (<https://www.nist.gov/>) Cryptographic Algorithm Validation Program (<https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program>) (CAVP). Please communicate questions and feedback concerning this document to the CAVP by opening an issue on the usnistgov/ACVP GitHub repository at <https://github.com/usnistgov/ACVP/issues>.

4. Conventions

4.1. Notation conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "*NOT RECOMMENDED*", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 of [RFC2119] and [RFC8174] when, and only when, they appear in all capitals, as shown here.

4.2. Terms and Definitions

4.2.1. Prompt

JSON sent from the server to the client describing the tests the client performs

Celi	Expires 18 October 2026	[Page 3]
Internet-Draft	ACVP ML-DSA	April 2026

4.2.2. Registration

The initial request from the client to the server describing the capabilities of one or several algorithm, mode and revision combinations

4.2.3. Response

JSON sent from the client to the server in response to the prompt

4.2.4. Test Case

An individual unit of work within a prompt or response

4.2.5. Test Group

A collection of test cases that share similar properties within a prompt or response

4.2.6. Test Vector Set

A collection of test groups under a specific algorithm, mode, and revision

4.2.7. Validation

JSON sent from the server to the client that specifies the correctness of the response

5. Supported ML-DSA Algorithms

The following ML-DSA algorithms MAY be advertised by the ACVP compliant cryptographic module. The list is in the form "algorithm / mode / revision".

- * ML-DSA / keyGen / FIPS204
- * ML-DSA / sigGen / FIPS204
- * ML-DSA / sigVer / FIPS204

6. Test Types and Test Coverage

Celi	Expires 18 October 2026	[Page 4]
Internet-Draft	ACVP ML-DSA	April 2026

6.1. Test Types

The ACVP server performs a set of tests on the specified ML-DSA algorithm in order to assess the correctness and robustness of the implementation. A typical ACVP validation session SHALL require multiple tests to be performed for every supported permutation of ML-DSA capabilities. This section describes the design of the tests used to validate implementations of the ML-DSA algorithms. The number of tests MAY vary but the minimum number required by each test type SHALL be included by the server. The test type describes the format of the test rather than the intention of the test. Multiple tests of the same test type MAY cover different assurances regarding the implementation.

6.1.1. ML-DSA KeyGen Test Types

- * ML-DSA / keyGen / * "AFT" - Algorithm Functional Test.

For each test case provided, the IUT SHALL generate a key pair from a provided seed. The key pair is communicated to the ACVP server and compared against the key pair generated by the server. This tests the implementation of Algorithm 6 ML-DSA.KeyGen_internal() from [FIPS204]. The server SHALL provide at least 25 tests for each combination of capabilities.

6.1.2. ML-DSA SigGen Test Types

- * ML-DSA / sigGen / * "AFT" - Algorithm Functional Test.

The IUT SHALL generate valid signatures based on the ACVP-provided message, mu (for external mu testing), private key, context (for external interface testing), hashAlg (for preHash testing), and randomness (for non-deterministic signature testing). The signature is then compared to the known signature by the ACVP server. This tests the implementation of Algorithm 2 ML-DSA.Sign(), Algorithm 4 HashML-DSA.Sign(), and Algorithm 7 ML-DSA.Sign_internal() from [FIPS204].

There are several assurances to obtain from AFTs. First is correctness of the algorithm implementation. The server SHALL include at least 15 tests for each combination of capabilities to meet this assurance. The "hashAlg" capability MAY be excluded from the combination of capabilities due to the number of supported hash functions. Each "hashAlg" provided by the IUT SHALL be covered within the, minimum, 15 tests generated for each combination of capabilities.

Celi

Expires 18 October 2026

[Page 5]

Internet-Draft

ACVP ML-DSA

April 2026

The second assurance is correctness under all rejection paths. Within ML-DSA sigGen, i.e., in Algorithm 7 ML-DSA.Sign_internal(), the algorithm enters a loop until a valid signature is found. The loop contains four potential reasons to reject a candidate signature: if the z infinity norm is too large, if the r infinity norm is too large, if there are too many hints in h, or if the ct0 infinity norm is too large. These conditions occur at various rarities that can make it difficult to test each error condition with randomized testing. If an implementation adheres strictly to the pseudocode in Algorithm 7 ML-DSA.Sign_internal() [FIPS204], the following table provides helpful known answer tests that trigger each rejection case exactly once. If the implementation varies from the pseudocode, it would be prudent to use a debugger or other tooling to ensure that all rejection cases are triggered by testing. To save on space, the table will provide the seed used to generate the key pair, a hash of the keys, SHA2-256(pk || sk), and a hash of the resulting signature, SHA2-256(sig). All cases are defined using the deterministic signature method, Algorithm 7 ML-DSA.Sign_internal(). Note that the ct0 infinity norm check only applies to ML-DSA-44. This condition is not possible on the other security levels. Thanks to Qinglai Xiao and Mike Hamburg (Rambus Inc) for providing code to generate these test cases; more information on their technique is available on the NIST PQC-Forum [PQCForum].

The server SHALL include at least 5 tests that cover all rejection outcomes for all combinations of capabilities where "signatureInterface" = "internal" and "deterministic" = true. The internal signature interface is described in [FIPS204] Algorithm 7 ML-DSA.Sign_internal() must be used to test this assurance. The Table 1 provides valid test cases for this assurance.

The third assurance is correctness under a large number of rejections. The standard, [FIPS204], does not define an upper limit on the number of potential rejections. Implementations that abort early may leak information about the underlying key. The server SHALL include at least 5 tests that lead to at least 32 rejections where "signatureInterface" = "internal" and "deterministic" = true. The internal signature interface described in [FIPS204] Algorithm 7 ML-DSA.Sign_internal() must be used to test this assurance. The Table 2 provides valid test cases for this assurance.

Security Level	Seed	Key Hash	M'	Signature Hash
ML-DSA-44	5C624FCC 18624524	AC825C59	951FDF54	DCC71A42
	52D0C665 840D8237	D8A4C453	73A4CBA6	1BC6FFAF
	F43108E5 499EDCDC	A2C4EFEA	D9E5B5DB	B7DF0C7F

Celi

Expires 18 October 2026

[Page 6]

Internet-Draft

ACVP ML-DSA

April 2026

	108FBC49 D596E4B7	8395741C	7E79FB81	6D018A19
		A404F300	73921BA5	ADA154D1
		0E28D56B	B13E9271	E2EE360E
		25D03BB4	401B8F90	D533CECD
		02E5CB2F	7B8B7D5B	5DC980AD

ML-DSA-44	836EABED B4D2CD9B E6A4D957 CF5EE6BF 48930413 6864C55C 2C5F01DA 5047D18B	E1FF40D9 6E3552FA B531D171 5084B7E3 8CCDBACC 0A8AF94C 30959FB4 C7F5A445	199A0AB7 35E90041 63DD02D3 19A61CFE 81638E3B F47BB1E9 0E90D6E3 EA545247	A2608BC2 7E60541D 27B6A14F 460D54A4 8C0298DC C3F45999 F29047A3 135C4941
ML-DSA-44	CA5A01E1 EA6552CB 5C980346 2B94C2F1 DC9D13BB 17A6ACE5 10D15705 6A2C6114	A4652DC4 A2710952 68DD84A5 B0744DFD BE2E642E 4D41FBC4 329C2FBA 534C0E13	8C8CACA8 8FFF52B9 33051053 7B3701B3 993F3726 136A650F 48F86045 51550832	B4B14220 9137397D AD504CAE D01D390A DAF49973 D8D2414F C3457FB7 AF775189
ML-DSA-44	9C005F15 50B4F318 55C6B92F 97873673 3F37791C B39DD182 D7BA5732 BDC2483E	2485AA99 345F1B33 4D4D94B6 10FBFFCC B626CBFD 4E9FF0E1 F6FC3509 3C423544	B744343F 30F7FEE0 88998BA5 74E799F1 BF3939C0 6C29BF9A C10F3588 A57E21E2	5B80A60B AA480B9D 0C7D2C05 B50928C4 BF6808DD A6936420 58A3EB77 EAA768FC
ML-DSA-44	4FAB5485 B009399E 8AE6FC3D 3EEFBFE8 E09796E4 477AABD5 EB1CC908 FA734DE3	CB56909A 7CF3008A 662DC635 EDCB79DC 151CA7AC BAE17B54 4384ABD9 1BBBC1E9	7CAB0FDC F4BEA5F0 39137478 AA45C9C4 8EF96D90 6FC49F6E 2F138111 BF1B4A4E	6CC38D73 D639682A BC556DC6 DCF436DE 24033091 F34004F4 10FABC68 87F77AB0
ML-DSA-65	464756A9 85E5DF03 739D95DD 309C1ED9 C5B04254 CC294E7E 7EB9B936 5EE15117	AE95EA0D AA80199E 7B4A74EB 5A1B1DC6 C3805BD0 1D2FA78D	491101BB A044DE6E 44A63796 C33CDA05 1BB05A60 725B87AF	8E08EA0C 8DB94168 5B9905A7 3B0B57BA D3500B1F 73490480

Celi

Expires 18 October 2026

[Page 7]

Internet-Draft

ACVP ML-DSA

April 2026

		7C4FBA8C 255AA13D	4BA9DB94 0C03AC09	B24375B4 1230CC04
ML-DSA-65	235A48DB 4CA7916B 884F424A 8586EFD5 17E87C64 AECEC0FC E9A3CC21 2BA1522E	1AC58A90 9DB4D7BC 2473AB5E 24AF7682 79C76F86 A82D4482 58E24EEA 4EA6B713	F8CE85CB 2EC474FF BF5A3FFA E029CE6F 4526B8D5 97655067 F97F438B 81071E9B	AE9531A0 1738615B 6D33C77B 3FF618A8 6E101FDC 4C850468 1F0EDFA6 4511AD63
ML-DSA-65	E13131B7 05A76030 5FEFFEBF E99082E2 691A444B BEFCC3ED F67D9098 86200207	B422093F 95CC489C 52F4FA2B 8973A2FD DD44426D 1D04D1AA	CD365512 C7E61BBA A130800B 37F3BB46 AAF1BEEF 3742EA8A	3C55E604 DECA7B89 A99305D7 A391C35F 66A17C19 23F46767

			EEFC8715 D417181F	9010A6DD 4576ED0B	5EC951C0 948D21C9
ML-DSA-65	0A4793E0 40A4BC0D 0F37643D 12C1EA1F 10648724 609936C7 6E0EC83E 37209E92		622D26D5 36D4D66C D94956B3 3A74E2E8 30ED265D 25C34FF7 C3E52434 03146ADF	6D9C7A79 5E48D80A 892CBF4D 45584297 87277E38 06EB5D0B CE1640EE BBBF9AEC	3B141110 B9F56540 B2D49AAC DE639997 4A4EAC40 621E367E 68D4504F 294DB21B
ML-DSA-65	F865B889 E5022D54 BABC81CA 67E7EB39 F1AC42F9 2CF5295C 3DA5C966 7DB1B924		45BC8EDD 1A620C46 E973E346 84427072 1824D978 88BC1742 81852D98 B7E8F4A3	047AFAAD BE020ED2 D766DA85 317DEDE8 0BE55054 5F0B21E3 F555A990 F8004258	56308A35 78360C41 356BA9C9 7D3240E0 1767FA76 BBBA9FD0 CC6CFA9A DD088DB9
ML-DSA-87	0D582191 32746BE0 77DFE821 E9F8FD87 857B28AB 91D6A567 E312A73E 2636032C		4D261270 341A7AC6 B66900DD C2B8AB34 AB483C89 7410DDF3 B2C072BD DA416434	3AA49EF7 2D010AEC 19383BA1 E83EC2DD 3DCC207A 96FFCEB9 FFA269E3 E3D66400	5049DC39 045618B9 03C71595 B3A3E07A 731F95D3 7304623A CC98BCEF 4258B4CA

Celi

Expires 18 October 2026

[Page 8]

Internet-Draft

ACVP ML-DSA

April 2026

ML-DSA-87	146C47AB 9F88408E B76A8132 94D533B2 9D7E0FDA 75DA5A4E 7C69EB61 EFEEBB78		05194438 AF855B79 DB8CCCCB 647D6BA5 C7AAF901 BBD09D3B 29395F0E A431D164	82C44F99 8A8D24F0 56084D0E 80ECFD84 34493385 A284C699 74923C27 0D397782	CFFC5988 A351E14A 3EE1282F 042A1436 79C45038 14296B27 993949A7 FF966F57
ML-DSA-87	049D9B0B 646A2AC7 F50B63CE 5E4BFE44 C9B87634 F4FF6C14 C513E388 B8A1F808		AC8FE6B2 FE26591B 129EA536 A9A001C7 85D8ACBD D9489F6E 51469A15 6E9E635D	FEBC9F8A E159002B E1A11D39 5959DD7F C2071813 5690CDAA 2BCFB580 1C02AB89	FF400608 9BDF7337 E868F86D DF48F239 D2A52EA1 D0F686E0 103BF19C 3B571DB1
ML-DSA-87	9823DDDE 446A8EA8 83DAD3AC 6477F798 39FDC2D2 DEF2416B E0A8B71C FBC3F5C6		525010E3 07C4EA76 67D54EE2 7007C219 B01F4CF8 8DC3AB2D E8E9AAA5 9440A884	F7592C97 C1A96A2F 4053588F 5CDAD4C5 0BF7C375 2709854F A27779B4 45DD2BA2	FD775760 2B83B0A6 7A314CD5 BCC880E7 AE47ACDF 4D6AF982 69028EFB 486838F7
ML-DSA-87	AE213FE8 589B414F 53780D8B 9B683717 9967E13C B474C5AD 365C0437 78D2BC90		D4988E91 064E5DF6 D867434D 1DED16DC	19C1913B A76FF045 96BB7CC8 0FD825A5	23FE743E DD101970 D499E7EB 57A7AA24

		D8533E39	AEDEF5D5	5BAF417E
		E420DC2B	AD61CEDB	851B260C
		4EB9E40A	5203E6D7	55DD525A
		84146F7D	EDB18877	445F08DA

Table 1: ML-DSA Algorithm 7 ML-DSA.Sign_internal() Known Answer Tests for Rejection Cases

Security Level	Rejection Count	Seed	Key Hash	M'	Signature Hash
ML-DSA-44	77	090D97C1	26D79E40	E3838364	CD91150C
		F4166EB3	68040E99	B37F47ED	610FF02D
		2CA67C5F	6BC9EB50	FCA2B577	E1DD7049
		B564ACBE	34C20489	B20B80C3	C309EFE8

Celi

Expires 18 October 2026

[Page 9]

Internet-Draft

ACVP ML-DSA

April 2026

		0735DB4A	C0AD38DC	CB51B9F5	00CE5C1B
		F4B8DB3A	2FEC1918	6E0E4CDB	C2E5A32D
		7C2CE740	D0760C86	7DF002C8	752AB62C
		2357CA44	21872408	74039252	5BF5E16F
ML-DSA-44	100	CFC73D07	89142AB2	0960C13E	B6296FFF
		A883543A	6D6EB6C0	9BA467A9	0C1F23DE
		804F7700	1FA3F189	38450120	4906D581
		70861825	A9C87759	CC96FF6F	44B00A2D
		143A62F2	7740D685	04B7E557	B13AD25E
		F97D05FC	983F29BB	C99A8386	49B4B857
		E00FD8B2	DD359664	19A48F9A	3A62EFEE
		5D29A43F	8266AE0E	38738AB8	CB544DD7
ML-DSA-65	64	26B605C7	5DA13E57	C9B07E7D	12A4637E
		8AC762FA	1DF80867	DC027446	3A833A5A
		1634C6F9	A8F27E0F	8F312F5C	2A46F6A9
		1DD117C4	F81BE725	692A54AC	91399E54
		FBFF7F3A	2A1ABF89	73D1E34D	4B62A230
		7E7781F0	B3D6A03D	8638E20A	B7AA82F7
		CC83B628	4036AF64	2CD3C788	366840FF
		1F04AD7F	3EFBB04B	F27D4355	6A88DE61
ML-DSA-65	73	9191CF38	7930D4E5	E616E36E	3D758ACE
		1BEE1747	2BA03B61	81AA1EC3	312433D7
		5C011986	DAA57743	92621094	80403B3D
		EFB6AFB1	B39E291D	21AE0DDD	4273171F
		EFA69974	824DC156	A5E3B5A8	B93D008B
		42FD3342	356C6B1A	F4A252BC	39535214
		7353F1DA	8232574D	A27AE882	2C6DC517
		1AA39FC0	5C8BDD08	538DF618	3E517310
ML-DSA-65	66	516912C7	0FD15951	9247CA75	E5CE2678
		B90A3DBE	B93A4D19	F9456226	00EDF335
		009B7478	446B48D4	A0C783DA	88451050
		DBCAF0F5	7D32D2CA	BCC33FF5	F9B4A5BF
		C5C9ED96	2253FF43	B4B48957	97030D04
		99A20D0C	BB8CCCB3	5ADED543	5132A7E3
		A56CC516	4C07E5F1	E74B29B4	ED9210E7
		E5A444CD	A3181B7A	5F9C8EF2	4028D23B
ML-DSA-65	65	D4B841F8	0039C128	17523165	8830EA25
		82D50AB9	DDE6923E	7B0F3C70	4AF2854B
		E590066B	A08FF14F	65947999	F67C2B90

AFABA0F0	5C5C66DC	467C3420	7E2321C9
D04D3264	B282B471	64F29BFA	4FD6EFB2
1C0B978E	FD1917DB	EB553E97	FDAA7766
54CCAA69	EBE07C8C	561407D5	9FC3A5C4

Celi Expires 18 October 2026 [Page 10]

Internet-Draft ACVP ML-DSA April 2026

		A6E8D2C4	45B73F8A	560E3AEB	426C57C9
ML-DSA-65	64	5492EB8D	573DCD99	33D2753E	84D4AF50
		811072C0	C86DAE81	D87D0003	933D6E13
		30A30CC6	F6F80CB0	B44C1AF5	D4332B86
		6B23A173	0AF40846	F72EB931	AF0692A6
		059EBA0D	028EA8F9	F559C6B4	6F5030AB
		4868CCB9	FE63102F	931AF7E2	01C2EAC4
		2FBE2510	E4A78238	49F65D3F	131A5EEB
		B4A5915F	BC7B660E	A7613295	F78CE9E5
ML-DSA-87	64	B5C07ECE	5D22F4C4	D1D5C2D1	54F0A9CB
		FE9E7C3B	0F6EEB96	67D6E629	26F98B39
		885FDEF0	BB891DB1	06790A5F	4A35918E
		32BDF9F8	5884ED4B	EDF5A0A7	CA6760EB
		07B4011E	0009EA02	54CFAF47	D10753FC
		2DFE6806	A24D9D1E	E6A11AEB	5CDBA8BE
		C088D208	9ADFC81C	93FB8C41	508873AD
		1631C8EB	7A42EA7F	934C31F8	83538131
ML-DSA-87	65	E8FC3C9F	B6C4DC9B	3B435F7A	E337495F
		AD711DDA	20CE5D0F	2CE431C7	08773F14
		2946334F	445931EE	AB8EAE09	FB26A3E2
		BBD33146	316CF067	91C5DAC6	29B9B26D
		8D6E9AB4	6E806D1A	10827C99	086644C7
		8EB86DCD	6A988688	D2780304	FDC30026
		03F300A1	81D060EA	6FBC6C56	7F9DCDD5
		7AEBCE5E	27CEB139	7D6B71F2	D78DB849
ML-DSA-87	64	151F8088	127972C3	C628CE94	99B552B2
		6D6CE8C3	3323FEFB	D2AA99AA	14325442
		B428964F	F6B69C19	50CF15B1	48BFF47A
		E02C40CA	E0C86F41	47D4F9A9	C8F24CB7
		0C8EFFA1	558D9AB2	C62A3D46	8DBB25C9
		00EE089E	B1A8AD6F	12152DE0	683F3ADC
		54D78534	39BD0A02	A502C377	B75614BE
		4FCCF719	45DC8D7E	F472D614	D58A0358
ML-DSA-87	64	48BEFFB4	72DA77CF	D2756A8F	E643914B
		C97E59E4	563CBB53	B4E47F79	8556D053
		74E1906F	0129F601	6AF704ED	60C65EB3
		39888BE5	29AF989C	0FC8C6E5	E7A06BE7
		AE62F6A0	A4036BA1	73D42DFA	C398B82D
		11C05EF6	058267BF	B443B329	49973EEF
		A6B8D1E5	BA34A2C7	F00F8DB2	DC711E65
		4F2171B7	0BE803C4	FF12C465	B11EB5E8
ML-DSA-87	69	FE2DA9DD	7422DBE3	A86B29AD	8D25818D

Celi Expires 18 October 2026 [Page 11]

Internet-Draft ACVP ML-DSA April 2026

93A077FC	F476FFE4	F2300D26	D972FFF5
----------	----------	----------	----------

		B6452AC8	1A4EFB33	36E21D4A	B9E9B4CC
		8D0A5762	F3DDFD8B	350CD18E	534A9510
		EB896BAA	328029BA	55A25437	0A1340C1
		AC6CE7D0	30506038	9C3659A7	C81D1486
		1CB1370B	66C36CFB	A95D8734	A68939D3
		A8322390	C2EE4B87	CEC1F005	40E0A58B

Table 2: ML-DSA Algorithm 7 ML-DSA.Sign_internal() Known Answer Tests for Number of Rejection Cases

6.1.3. ML-DSA SigVer Test Types

- * ML-DSA / sigVer / * "AFT" - Algorithm Functional Test.

The IUT SHALL determine the validity of the signature based on the ACVP-provided message, mu (for external mu testing), context (for external interface testing), hashAlg (for preHash testing), public key, and signature. This tests the implementation of Algorithm 3 ML-DSA.Verify(), Algorithm 5 HashML-DSA.Verify(), and Algorithm 8 ML-DSA.Verify_internal() from [FIPS204]. Tests for signature verification are performed by the server modifying a valid signature to obtain specific assurances from the implementation. The server SHALL include at least 3 tests for each modification type (including "valid signature") for all combinations of capabilities. The "hashAlg" capability MAY be excluded from the combination of capabilities due to the number of supported hash functions. Each "hashAlg" provided by the IUT SHALL be covered within the, minimum, 15 tests generated for each combination of capabilities.

The signature modifications are:

- * "valid signature and message - signature should verify successfully" - No modification is made and the signature is valid.
- * "modified message" - The message that was signed has been changed. The signature is not valid.
- * "modified signature - commitment" - A component of the signature, the bytes that commit the signer to the message, has been changed. The signature is not valid.
- * "modified signature - z" - A component of the signature, the bytes that allow the verifier to construct the vector z, has been changed. The signature is not valid.

Celi Expires 18 October 2026 [Page 12]

Internet-Draft ACVP ML-DSA April 2026

- * "modified signature - hint" - A component of the signature, the bytes that allow the verifier to construct the hint array, has been changed. The signature is not valid.

6.2. Test Coverage

The tests described in this document have the intention of ensuring an implementation is conformant to [FIPS204].

6.2.1. Requirements Covered

- * The tests will ensure conformity and correctness of an implementation of the algorithms supported.

6.2.2. Requirements Not Covered

- * FIPS 204 Section 3.5. Additional Requirements. Requirements outlined in this section are not testable by an ACVP server. An ACVP server will not test the zeroization of intermediate values, security strength of the deterministic random bit generators (DRBGs), or incorrect length signatures or public keys.

7. Capabilities Registration

ACVP requires crypto modules to register their capabilities. This allows the crypto module to advertise support for specific algorithms, notifying the ACVP server which algorithms need test vectors generated for the validation process. This section describes the constructs for advertising support of ML-DSA algorithms to the ACVP server.

The algorithm capabilities MUST be advertised as JSON objects within the 'algorithms' value of the ACVP registration message. The 'algorithms' value is an array, where each array element is an individual JSON object defined in this section. The 'algorithms' value is part of the 'capability_exchange' element of the ACVP JSON registration message. See the ACVP specification [ACVP] for more details on the registration message.

7.1. Prerequisites

Each algorithm implementation MAY rely on other cryptographic primitives. For example, RSA Signature algorithms depend on an underlying hash function. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a mechanism for specifying the required prerequisites:

Prerequisites, if applicable, MUST be submitted in the registration as the prereqVals JSON property array inside each element of the algorithms array. Each element in the prereqVals array MUST contain the following properties

JSON Property	Description	JSON Type
algorithm	a prerequisite algorithm	string
valValue	algorithm validation number	string

Table 3: Prerequisite Properties

A "valValue" of "same" SHALL be used to indicate that the prerequisite is being met by a different algorithm in the capability exchange in the same registration.

An example description of prerequisites within a single algorithm capability exchange looks like this

```
"prereqVals":
[
  {
    "algorithm": "Alg1",
    "valValue": "Val-1234"
```

```

    },
    {
      "algorithm": "Alg2",
      "valValue": "same"
    }
  ]

```

7.2. Required Prerequisite Algorithms for ML-DSA Validations

Each ML-DSA implementation relies on other cryptographic primitives. For example, ML-DSA keyGen uses an underlying SHA algorithm. Each of these underlying algorithm primitives must be validated, either separately or as part of the same submission. ACVP provides a mechanism for specifying the required prerequisites:

Celi Expires 18 October 2026 [Page 14]
 Internet-Draft ACVP ML-DSA April 2026

JSON Value	Description	JSON Type	Valid Values
algorithm	a prerequisite algorithm	string	SHA, or DRBG
valValue	algorithm validation number	string	Actual number or "same"
prereqAlgVal	prerequisite algorithm validation	object with algorithm and valValue properties	See above

Table 4: Required ML-DSA Prerequisite Algorithms JSON Values

7.3. ML-DSA keyGen Registration Properties

Each ML-DSA keyGen algorithm capability advertised is a self-contained JSON object using the following values.

JSON Value	Description	JSON Type	Valid Values
algorithm	The ML-DSA algorithm to be validated	string	"ML-DSA"
mode	The ML-DSA mode to be validated	string	"keyGen"
revision	The algorithm testing revision to use	string	"FIPS204"
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 7.2

parameterSets	The ML-DSA parameter sets supported	array of strings	"ML-DSA-44", "ML-DSA-65", "ML-DSA-87"
---------------	-------------------------------------	------------------	---

Table 5: ML-DSA keyGen Algorithm Capabilities JSON Values

7.3.1. ML-DSA keyGen Mode Capabilities Example

Below is an example of the registration for ML-DSA / keyGen / FIPS204

```
{
  "algorithm": "ML-DSA",
  "mode": "keyGen",
  "revision": "FIPS204",
  "prereqVals": [
    {
      "algorithm": "SHA",
      "valValue": "123456"
    }
  ],
  "parameterSets": ["ML-DSA-44", "ML-DSA-65", "ML-DSA-87"]
}
```

7.4. ML-DSA sigGen Registration Properties

Each ML-DSA sigGen algorithm capability advertised is a self-contained JSON object using the following values.

JSON Value	Description	JSON Type	Valid Values
algorithm	The ML-DSA algorithm to be validated	string	"ML-DSA"
mode	The ML-DSA mode to be validated	string	"sigGen"
revision	The algorithm testing revision to use	string	"FIPS204"
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 7.2
capabilities	An array of individual capability objects	array of capability objects	See Table 7
deterministic	The ML-DSA signature generation modes supported	array of booleans	true, false
signatureInterfaces	The ML-DSA signature interfaces supported	array of strings	"internal", "external"
preHash	Whether ML-DSA and HashML-DSA are supported. Only required when "signatureInterfaces" contains "external".	array of strings	"pure", "preHash"
externalMu	Whether mu is computed internally to the module or externally from the module under test. Only applicable when "signatureInterfaces" contains "internal".	array of booleans	true, false

Table 6: ML-DSA sigGen Algorithm Capabilities JSON Values

The capability objects use the following properties.

Celi

Expires 18 October 2026

[Page 17]

Internet-Draft

ACVP ML-DSA

April 2026

JSON Value	Description	JSON Type	Valid Values
parameterSets	The ML-DSA parameter sets supported	array of strings	"ML-DSA-44", "ML-DSA-65", "ML-DSA-87"
messageLength	The	domain	Min: 8, Max: 65536,

	supported message lengths in bits		Incr: 8
hashAlgs	The hash algorithms used if preHash ML-DSA is used	array of strings	"SHA2-224", "SHA2-256", "SHA2-384", "SHA2-512", "SHA2-512/224", "SHA2-512/256", "SHA3-224", "SHA3-256", "SHA3-384", "SHA3-512", "SHAKE-128", "SHAKE-256"
contextLength	The supported context lengths if the external interface is used	domain	Min: 0, Max: 2040, Incr: 8

Table 7: ML-DSA sigGen Capability Property JSON Values

7.4.1. ML-DSA sigGen Mode Capabilities Example

Below is an example of the registration for ML-DSA / sigGen / FIPS204

```
{
  "algorithm": "ML-DSA",
  "mode": "sigGen",
  "revision": "FIPS204",
  "prereqVals": [
    {
      "algorithm": "SHA",
      "valValue": "123456"
    }
  ]
}
```

Celi

Expires 18 October 2026

[Page 18]

Internet-Draft

ACVP ML-DSA

April 2026

```
],
"capabilities": [
  {
    "parameterSets": [
      "ML-DSA-44",
      "ML-DSA-65",
      "ML-DSA-87"
    ],
    "messageLength": [
      {
        "min": 8,
        "max": 65536,
        "increment": 8
      }
    ],
    "hashAlgs": [
      "SHA2-224",
      "SHA2-256",
      "SHA2-384",
      "SHA2-512",
      "SHA2-512/224",
      "SHA2-512/256",
      "SHA3-224",

```

```

        "SHA3-256",
        "SHA3-384",
        "SHA3-512",
        "SHAKE-128",
        "SHAKE-256"
    ],
    "contextLength": [
        {
            "min": 0,
            "max": 2040,
            "increment": 8
        }
    ]
},
"deterministic": [
    true,
    false
],
"externalMu": [
    true,
    false
],
"signatureInterfaces": [
    "external",

```

Celi Expires 18 October 2026

[Page 19]

Internet-Draft

ACVP ML-DSA

April 2026

```

        "internal"
    ],
    "preHash": [
        "pure",
        "preHash"
    ]
}

```

7.5. ML-DSA sigVer Registration Properties

Each ML-DSA sigVer algorithm capability advertised is a self-contained JSON object using the following values.

Celi Expires 18 October 2026 [Page 20]
 Internet-Draft ACVP ML-DSA April 2026

JSON Value	Description	JSON Type	Valid Values
algorithm	The ML-DSA algorithm to be validated	string	"ML-DSA"
mode	The ML-DSA mode to be validated	string	"sigVer"
revision	The algorithm testing revision to use	string	"FIPS204"
prereqVals	Prerequisite algorithm validations	array of prereqAlgVal objects	See Section 7.2
capabilities	An array of individual capability objects	array of capability objects	See Table 9
signatureInterfaces	The ML-DSA signature interfaces supported	array of strings	"internal", "external"
preHash	Whether ML-DSA and HashML-DSA are supported. Only required when "signatureInterfaces" contains "external".	array of strings	"pure", "preHash"
externalMu	Whether mu is computed internally to the module or externally from the module under test. Only applicable when "signatureInterfaces" contains "internal".	array of booleans	true, false

Table 8: ML-DSA sigVer Algorithm Capabilities JSON Values

The capability objects use the following properties.

JSON Value	Description	JSON Type	Valid Values
parameterSets	The ML-DSA parameter sets supported	array of strings	"ML-DSA-44", "ML-DSA-65", "ML-DSA-87"
messageLength	The supported message lengths in bits	domain	Min: 8, Max: 65536, Incr: 8
hashAlgs	The hash algorithms used if preHash ML-DSA is used	array of strings	"SHA2-224", "SHA2-256", "SHA2-384", "SHA2-512", "SHA2-512/224", "SHA2-512/256", "SHA3-224", "SHA3-256", "SHA3-384", "SHA3-512", "SHAKE-128", "SHAKE-256"
contextLength	The supported context lengths if the external interface is used	domain	Min: 0, Max: 2040, Incr: 8

Table 9: ML-DSA sigVer Capability Property JSON Values

7.5.1. ML-DSA sigVer Mode Capabilities Example

Below is an example of the registration for ML-DSA / sigVer / FIPS204

```
{
  "algorithm": "ML-DSA",
  "mode": "sigVer",
  "revision": "FIPS204",
  "prereqVals": [
    {
      "algorithm": "SHA",
      "valValue": "123456"
    }
  ]
}
```

```
],
"capabilities": [
  {
    "parameterSets": [
      "ML-DSA-44",
      "ML-DSA-65",
```

```

    "ML-DSA-87"
  ],
  "messageLength": [
    {
      "min": 8,
      "max": 65536,
      "increment": 8
    }
  ],
  "hashAlgs": [
    "SHA2-224",
    "SHA2-256",
    "SHA2-384",
    "SHA2-512",
    "SHA2-512/224",
    "SHA2-512/256",
    "SHA3-224",
    "SHA3-256",
    "SHA3-384",
    "SHA3-512",
    "SHAKE-128",
    "SHAKE-256"
  ],
  "contextLength": [
    {
      "min": 0,
      "max": 2040,
      "increment": 8
    }
  ]
},
"externalMu": [
  true,
  false
],
"signatureInterfaces": [
  "external",
  "internal"
],
"preHash": [
  "pure",

```

Celi Expires 18 October 2026

[Page 23]

Internet-Draft

ACVP ML-DSA

April 2026

```

    "preHash"
  ]
}

```

8. Test Vectors

The ACVP server provides test vectors to the ACVP client, which are then processed and returned to the ACVP server for validation. A typical ACVP validation test session would require multiple test vector sets to be downloaded and processed by the ACVP client. Each test vector set represents an individual cryptographic algorithm defined during the capability exchange. This section describes the JSON schema for a test vector set used with Module Lattice-based Digital Signature Algorithm (ML-DSA) algorithms.

The test vector set JSON schema is a multi-level hierarchy that contains meta data for the entire vector set as well as individual test vectors to be processed by the ACVP client. The following table describes the JSON elements at the top level of the hierarchy.

JSON Values	Description	JSON Type
acvVersion	Protocol version identifier	string
vsId	Unique numeric vector set identifier	integer
algorithm	Algorithm defined in the capability exchange	string
mode	Mode defined in the capability exchange	string
revision	Protocol test revision selected	string
testGroups	Array of test group JSON objects. Depending on the algorithm, see Section 8.1.1, Section 8.2.1 or Section 8.3.1	array

Table 10: Top Level Test Vector JSON Elements

An example of this would look like this

Celi Expires 18 October 2026 [Page 24]
Internet-Draft ACVP ML-DSA April 2026

```
[
  {
    "acvVersion": <version>
  },
  {
    "vsId": 1,
    "algorithm": "Alg1",
    "mode": "Mode1",
    "revision": "Revision1.0",
    "testGroups": [ ... ]
  }
]
```

8.1. ML-DSA keyGen Test Vectors

8.1.1. ML-DSA keyGen Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the ML-DSA JSON elements of the Test Group JSON object.

The test group for ML-DSA / keyGen / FIPS204 is as follows:

JSON Value	Description	JSON type
------------	-------------	-----------

tgId	Numeric identifier for the test group, unique across the entire vector set	integer
testType	The test operation performed	string
parameterSet	The ML-DSA parameter set used	string
tests	Array of individual test vector JSON objects, which are defined in Section 8.1.2	array

Table 11: ML-DSA keyGen Test Group JSON Object

Celi Expires 18 October 2026 [Page 25]
Internet-Draft ACVP ML-DSA April 2026

8.1.2. ML-DSA keyGen Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA test vector.

JSON Value	Description	JSON type
tcId	Numeric identifier for the test case, unique across the entire vector set	integer
seed	The seed used to generate the key pair	hex

Table 12: ML-DSA keyGen Test Case JSON Object

The following is an example JSON object sent from the server to the client for ML-DSA / keyGen / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 1564,
    "algorithm": "ML-DSA",
    "mode": "keyGen",
    "revision": "FIPS204",
    "testGroups": [
      {
        "tgId": 1,
        "testType": "AFT",
        "parameterSet": "ML-DSA-44",
        "tests": [
          {
            "tcId": 1,
            "seed": "C105DC2..."
          }
        ]
      }
    ]
  }
]
```

8.2. ML-DSA sigGen Test Vectors

8.2.1. ML-DSA sigGen Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the ML-DSA JSON elements of the Test Group JSON object.

The test group for ML-DSA / sigGen / FIPS204 is as follows:

JSON Value	Description	JSON
------------	-------------	------

		type
tgId	Numeric identifier for the test group, unique across the entire vector set	integer
testType	The test operation performed	string
parameterSet	The ML-DSA parameter set used	string
deterministic	Whether the signatures should be generated using the deterministic or non-deterministic routine	boolean
signatureInterface	Whether the signature is generated using the internal or external routine	string
preHash	Whether the signature is generated using the prehash or pure routine	string
externalMu	Whether mu is computed internally or externally to the module under test	string
tests	Array of individual test vector JSON objects, which are defined in Section 8.2.2	array

Table 13: ML-DSA sigGen Test Group JSON Object

NOTE: The "preHash" property will only be present if "signatureInterface" is set to "external". The "externalMu" property will only be present if "signatureInterface" is set to "internal".

8.2.2. ML-DSA sigGen Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA test vector.

Celi Expires 18 October 2026 [Page 28]
Internet-Draft ACVP ML-DSA April 2026

JSON Value	Description	JSON type
tcId	Numeric identifier for the test case, unique across the entire vector set	integer
message	The message used to generate the signature	hex
mu	The mu used to generate the signature	hex
sk	When the test group property "testType": "AFT", the secret key that should be used to generate a signature	hex

rnd	When the test group properties "testType": "AFT" and "deterministic": false, the random value used to generate the signature	hex
context	When the test group property "signatureInterface": "external", the context used to construct the signed message	hex
hashAlg	When the test group property "preHash": "preHash", the hash function used to construct the signed message	string

Table 14: ML-DSA sigGen Test Case JSON Object

NOTE: When "signatureInterface" is set to "internal" for the test group, the "message" will only be present when "externalMu" is false, while "mu" will only be present when "externalMu" is true.

The following is an example JSON object sent from the server to the client for ML-DSA / sigGen / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "algorithm": "ML-DSA",
    "mode": "sigGen",
    "revision": "FIPS204",
    "testGroups": [
```

Celi

Expires 18 October 2026

[Page 29]

Internet-Draft

ACVP ML-DSA

April 2026

```
{
  "tgId": 1,
  "testType": "AFT",
  "parameterSet": "ML-DSA-44",
  "signatureInterface": "internal",
  "externalMu": false,
  "deterministic": true,
  "tests": [
    {
      "tcId": 1,
      "sk": "D4E36A5...",
      "message": "0D873AEFD..."
    }
  ]
},
{
  "tgId": 2,
  "testType": "AFT",
  "parameterSet": "ML-DSA-44",
  "signatureInterface": "internal",
  "externalMu": true,
  "deterministic": false,
  "tests": [
    {
      "tcId": 71,
      "sk": "D4E36A5...",
      "mu": "34F4B...",
      "rnd": "7AFF2F22A..."
    }
  ]
}
```



```

    }
  ],
  {
    "tgId": 3,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "external",
    "preHash": "pure",
    "deterministic": true,
    "tests": [
      {
        "tcId": 71,
        "sk": "D4E36A5...",
        "message": "34F4B...",
        "context": "ABCD"
      }
    ]
  },
  {

```

Celi

Expires 18 October 2026

[Page 30]

Internet-Draft

ACVP ML-DSA

April 2026

```

    "tgId": 4,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "external",
    "preHash": "preHash",
    "deterministic": false,
    "tests": [
      {
        "tcId": 71,
        "sk": "D4E36A5...",
        "message": "34F4B...",
        "rnd": "7AFF2F22A...",
        "context": "ABCD",
        "hashAlg": "SHA2-256"
      }
    ]
  }
]
}
]
}
]

```

8.3. ML-DSA sigVer Test Vectors

8.3.1. ML-DSA sigVer Test Groups JSON Schema

The testGroups element at the top level in the test vector JSON object is an array of test groups. Test vectors are grouped into similar test cases to reduce the amount of data transmitted in the vector set. For instance, all test vectors that use the same key size would be grouped together. The Test Group JSON object contains meta data that applies to all test vectors within the group. The following table describes the ML-DSA JSON elements of the Test Group JSON object.

The test group for ML-DSA / sigVer / FIPS204 is as follows:

Celi Expires 18 October 2026 [Page 31]
 Internet-Draft ACVP ML-DSA April 2026

JSON Value	Description	JSON type
tgId	Numeric identifier for the test group, unique across the entire vector set	integer
testType	The test operation performed	string
parameterSet	The ML-DSA parameter set used	string
signatureInterface	Whether the signature is generated using the internal or external routine	string
preHash	Whether the signature is generated using the prehash or pure routine	string
tests	Array of individual test vector JSON objects, which are defined in Section 8.3.2	array

Table 15: ML-DSA sigVer Test Group JSON Object

NOTE: The "preHash" property will only be present if "signatureInterface" is set to "external". The "externalMu" property will only be present if "signatureInterface" is set to "internal".

8.3.2. ML-DSA sigVer Test Case JSON Schema

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA test vector.

Celi Expires 18 October 2026 [Page 32]
 Internet-Draft ACVP ML-DSA April 2026

JSON Value	Description	JSON type
tcId	Numeric identifier for the test case, unique across the entire vector set	integer
pk	The public key used to verify the signature	hex
message	The message used to verify with the signature	hex
mu	The mu used to generate the signature	hex
signature	The signature to verify	hex
context	When the test group property "signatureInterface": "external", the context used to construct the signed message	hex
hashAlg	When the test group property "preHash": "preHash", the hash function used to construct the signed message	string

Table 16: ML-DSA sigVer Test Case JSON Object

NOTE: When "signatureInterface" is set to "internal" for the test group, the "message" will only be present when "externalMu" is false, while "mu" will only be present when "externalMu" is true.

The following is an example JSON object sent from the server to the client for ML-DSA / sigVer / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "algorithm": "ML-DSA",
    "mode": "sigVer",
```

Celi

Expires 18 October 2026

[Page 33]

Internet-Draft

ACVP ML-DSA

April 2026

```
"revision": "FIPS204",
"isSample": false,
"testGroups": [
  {
    "tgId": 1,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "internal",
    "externalMu": false,
    "tests": [
      {
```

```

        "tcId": 1,
        "pk": "3FE65294...",
        "message": "4F0D7...",
        "signature": "C29A11B6C..."
    },
    {
        "tcId": 2,
        "pk": "3FE65294...",
        "message": "84793...",
        "signature": "6A99215FC2..."
    }
]
},
{
    "tgId": 2,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "internal",
    "externalMu": true,
    "tests": [
        {
            "tcId": 3,
            "pk": "3FE65294...",
            "mu": "4F0D7...",
            "signature": "C29A11B6C..."
        }
    ]
},
{
    "tgId": 3,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "external",
    "preHash": "pure",
    "tests": [
        {
            "tcId": 4,

```

Celi

Expires 18 October 2026

[Page 34]

Internet-Draft

ACVP ML-DSA

April 2026

```

        "pk": "4C7029A9...",
        "message": "75E...",
        "context": "548...",
        "signature": "04D1B8..."
    },
]
},
{
    "tgId": 4,
    "testType": "AFT",
    "parameterSet": "ML-DSA-44",
    "signatureInterface": "external",
    "preHash": "preHash",
    "tests": [
        {
            "tcId": 5,
            "pk": "5FB42D9CF5EE...",
            "message": "8740606...",
            "context": "0710CE7...",
            "hashAlg": "SHA2-224",
            "signature": "C0D60..."
        }
    ]
}
}

```

```
]
}
]
```

9. Test Vector Responses

After the ACVP client downloads and processes a vector set, it must send the response vectors back to the ACVP server. The following table describes the JSON object that represents a vector set response.

JSON Property	Description	JSON Type
acvVersion	The ACVP version used	string
vsId	The vector set identifier	integer
testGroups	The test group objects in the response, see Table 18	array

Table 17: Response JSON Object

An example of this is the following

```
{
  "acvVersion": "version",
  "vsId": 1,
  "testGroups": [ ... ]
}
```

The 'testGroups' section is used to organize the ACVP client response in a similar manner to how it distributes vectors.

JSON Property	Description	JSON Type
tgId	The test group identifier	integer
tests	The test case objects in the response, depending on the algorithm see Table 19, Table 20, or Table 21	array

Table 18: Response Group Objects

An example of this is the following

```
{
  "tgId": 1,
  "tests": [ ... ]
}
```

9.1. ML-DSA keyGen Test Vector Responses

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA / keyGen / FIPS204 test vector.

JSON Value	Description	JSON type
tcId	The test case identifier	integer
pk	The computed public key	hex
sk	The computed secret key	hex

Table 19: ML-DSA keyGen Test Case Response JSON Object

The following is an example JSON test vector response object for ML-DSA / keyGen / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "pk": "1012798...",
            "sk": "1012798..."
          }
        ]
      }
    ]
  }
]
```

9.2. ML-DSA sigGen Test Vector Responses

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA / sigGen / FIPS204 test vector.

JSON Value	Description	JSON type
tcId	The test case identifier	integer
signature	The generated signature	hex

Table 20: ML-DSA sigGen Test Case Response JSON Object

The following is an example JSON test vector response object for ML-DSA / sigGen / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 1564,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "signature": "D7D4275..."
          }
        ]
      }
    ]
  }
]
```

9.3. ML-DSA sigVer Test Vector Responses

Each test group contains an array of one or more test cases. Each test case is a JSON object that represents a single test vector to be processed by the ACVP client. The following table describes the JSON elements for each ML-DSA / sigVer / FIPS204 test vector.

JSON Value	Description	JSON type
tcId	The test case identifier	integer
testPassed	Whether or not the signature verified	boolean

Table 21: ML-DSA sigVer Test Case Response JSON Object

The following is an example JSON test vector response object for ML-DSA / sigVer / FIPS204.

```
[
  {
    "acvVersion": <acvp-version>
  },
  {
    "vsId": 0,
    "testGroups": [
      {
        "tgId": 1,
        "tests": [
          {
            "tcId": 1,
            "testPassed": false
          }
        ]
      }
    ]
  }
]
```

10. Security Considerations

There are no additional security considerations outside of those outlined in the ACVP document.

11. IANA Considerations

This document does not require any action by IANA.

12. Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", RFC 2119, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.

Celi	Expires 18 October 2026	[Page 39]
Internet-Draft	ACVP ML-DSA	April 2026

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", RFC 8174, RFC 8174, DOI 10.17487/RFC8174, May 2017, <<https://www.rfc-editor.org/info/rfc8174>>.

[FIPS204] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard", NIST FIPS 204, August 2024, <<https://csrc.nist.gov/pubs/fips/204/final>>.

[ACVP] Fussell, B., Vassilev, A., and H. Booth, "Automatic Cryptographic Validation Protocol", ACVP, 1 July 2019.

[PQCForum] "PQC-Forum", PQCForum, <<https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/G8Zf0hC-uu0/m/Kb3qNJb0AwAJ>>.

Author's Address

Christopher Celi (editor)
Email: christopher.celi@nist.gov

Celi

Expires 18 October 2026

[Page 40]